



CSE 351 Programming Languages Term Project Report

YU-Assembly Compiler

Ege Huriel

Assigned by Asst. Prof. Dr. Gizem Terci

June 8, 2026

Contents

Contents	2
1. Project Overview	4
1.1 Introduction	4
1.2 Objectives	4
1.3 Project Summary	4
2. YU Assembly Language Specification	5
2.1 Instruction Set	5
2.2 Language Rules	5
2.3 Memory Addressing	6
2.4 Valid and Invalid Programs	6
3. System Architecture	7
3.1 Overall Pipeline	7
3.2 Project Structure	7
3.3 Build System	7
4. BNF Grammar	8
5. Lexical Analysis	9
5.1 Token Definitions	9
5.2 Lexer Implementation:	9
5.3 Error Handling	9
6. Syntax Analysis	10
6.1 Grammar Rules	10
6.2 Parser Actions	10
6.3 Error Detection	10
7. Abstract Syntax Tree	11
7.1 Node Types	11
7.2 AST Construction	11
8. Loop Unrolling Optimization	12
8.1 Algorithm Overview	12
8.2 Backward Branch Detection	12
8.3 Example	12
9. Symbol Table — Bonus	13
9.1 Purpose	13
9.2 Two-Pass Algorithm	13

9.3 Implementation Details	13
10. Test Cases.....	14
10.1 Valid Programs.....	14
10.2 Invalid Programs	15
11. Compilation and Execution.....	16
12. Conclusion	17
13. References.....	18

1. Project Overview

1.1 Introduction

This report documents the design and implementation of a complete compiler front-end for YU assembly, a custom pseudo-assembly language developed as part of the CSE 351 Programming Languages. The project encompasses the full pipeline of a compiler front-end. Lexical analysis, syntax analysis, abstract syntax tree construction, a semantic analysis pass in the form of a symbol table and a code optimization pass implementing loop unrolling.

The YU assembly language was designed specifically for this project as a minimal but expressive pseudo-assembly language. It features a concise instruction set covering data movement, arithmetic bitwise operations and conditional branching. Programs written in YU assembly are statically analyzed and optimized by the compiler which produces a semantically equivalent but more efficient version of the original program.

The implementation is built in C using Flex for lexical analysis and Bison for syntax analysis, following classical compiler construction principles. The abstract syntax tree serves as the central data structure connecting all stages of the pipeline and all optimizations operate directly on this tree rather than on raw source text.

1.2 Objectives

The objectives of this project are organized around the three main tasks defined in the project handout:

1. Design the formal syntax of YU assembly using BNF grammar notation, defining all valid instruction forms, operand types and structural rules of the language.
2. Implement a functional lexer and parser using Flex and Bison that tokenizes input source files, validates their grammatical structure and constructs a fully typed Abstract Syntax Tree.
3. Implement a loop unrolling optimization pass that detects loops through backward branch analysis and duplicates the loop body by a factor of two, reducing the total number of branch and counter-increment instructions at runtime.
4. As a bonus objective, implement a symbol table that performs semantic analysis by validating that all branch instruction targets refer to labels that have been declared within the program.

1.3 Project Summary

The final implementation successfully satisfies all four objectives listed above. The compiler accepts any syntactically valid YU Assembly program, rejects invalid programs with descriptive error messages and produces an optimized output that is semantically equivalent to the original. The symbol table bonus provides an additional layer of correctness checking beyond what is guaranteed by the parser alone.

The project is organized into a modular directory structure with clearly separated source files a build directory for general artifacts, a binary output directory and a test suite covering both valid and invalid input scenarios.

2. YU Assembly Language Specification

2.1 Instruction Set

YU Assembly provides eleven instructions covering the fundamental operations required for assembly-level programming. The complete instruction set is summarized in the table below.

Instruction	Syntax	Description
mov	mov Rd, imm	Load immediate value into register Rd
load	load Rd, mem[addr]	Load value from memory address into Rd
store	store Rs, mem[addr]	Store register Rs to memory address
addi	addi Rd, Rs, imm	$Rd = Rs + \text{immediate}$
add	add Rd, Rs1, Rs2	$Rd = Rs1 + Rs2$
and	and Rd, Rs1, Rs2	$Rd = Rs1 \& Rs2$ (bitwise AND)
or	or Rd, Rs1, Rs2	$Rd = Rs1 Rs2$ (bitwise OR)
xor	xor Rd, Rs1, Rs2	$Rd = Rs1 \wedge Rs2$ (bitwise XOR)
blt	blt Rs1, Rs2, label	Branch if $Rs1 < Rs2$
bgt	bgt Rs1, Rs2, label	Branch if $Rs1 > Rs2$
beq	beq Rs1, Rs2, label	Branch if $Rs1 == Rs2$

2.2 Language Rules

The YU Assembly language enforces the following structural and semantic constraints, all of which are checked by the compiler:

1. Only registers R0 through R7 are valid. Any use of R8 or a higher-numbered register is detected by the lexer and reported as an error before parsing begins.
2. The mov instruction requires a register as its first operand. Using a label in the first operand position of mov is a syntax error that is explicitly caught and reported by the parser.
3. All instructions strictly require commas between operands. A missing comma is treated as a syntax error and reported with the line number where the violation occurred.
4. Branch instructions (blt, bgt, beq) take a label as their third operand. Using a register in this position is a syntax error.
5. Programs may contain multiple labeled blocks and branch instructions referencing different labels. The parser accepts arbitrarily complex multi-loop programs.
6. Programs without any loops are accepted and passed through unchanged by the optimization pass.
7. Bitwise instructions (and, or, xor) can be chained freely in sequence.

2.3 Memory Addressing

The load and store instructions support three memory addressing forms. All three forms are parsed and represented in the AST:

- `mem[R]` - base register only, no offset
- `mem[R+N]` - base register with positive integer offset
- `mem[R-N]` - base register with negative integer offset

In the AST, memory nodes store three fields: the base register number, the offset value and a sign indicator (0 for no offset, 1 for positive, -1 for negative). This representation is used by the loop unrolling pass to correctly adjust memory addresses in duplicated loop body instructions.

2.4 Valid and Invalid Programs

The following examples illustrate what the compiler accepts and rejects:

<code>mov R2, 0</code>	<code>mov R8, 5</code>	<code># Lexer error: invalid register</code>
<code>mov R3, 100</code>		
<code>mov R4, 50</code>		Invalid - R8 register
<code>L: load R1, mem[R2]</code>		
<code>store R1, mem[R3]</code>	<code>mov L, 5</code>	<code># Syntax error: register required</code>
<code>addi R2, R2, 1</code>		Invalid - label in mov operand
<code>addi R3, R3, 1</code>		
<code>blt R2, R4, L</code>	<code>mov R0 5</code>	<code># Syntax error: comma expected</code>

Valid - basic copy loop

Invalid - missing comma

3. System Architecture

3.1 Overall Pipeline

The compiler is structured as a sequential pipeline where the output each stage feeds directly into the next. The stages are:

1. Lexical Analysis (Flex) - The raw input file is read and converted into a stream of tokens. Invalid characters and out-of-range registers are caught and reported here.
2. Syntax Analysis (Bison) - The token stream is validated against the YU Assembly grammar. Valid programs result in an Abstract Syntax Tree; invalid programs produce descriptive error messages.
3. Symbol Table Check - The AST is traversed in two passes: first to collect all defined labels, then to validate all branch targets against the collected labels. Undefined targets are reported as semantic errors.
4. Loop Unrolling - The AST is flattened into a linear statement array, loops are detected by identifying backward branches and the loop body is duplicated by a factor of two with appropriate offset adjustments.
5. Output - The final optimized AST is traversed and printed to standard output in valid YU Assembly dyntax.

3.2 Project Structure

The project follows a clean directory structure separating source code, build artifacts and test files:

```
Yu-Assembly/  
  src/  
    lexer.l      - Flex lexical analyzer  
    parser.y     - Bison grammar and parser  
    ast.c / ast.h - AST node types and constructors  
    unroll.c / .h - Loop unrolling optimization  
    symtable.c / .h - Symbol table (bonus)  
    main.c       - Entry point and pipeline driver  
  build/         - Generated: lex.yy.c, parser.tab.c/h  
  bin/           - Compiled binary: yuasm  
  bnf/           - BNF grammar document  
  tests/         - Valid and invalid test programs  
  Makefile  
  README.md
```

3.3 Build System

The project uses a Makefile to automate the build process. Running make all performs three steps in the correct dependency order: Bison is invoked on parser.y to generate parser.tab.c and parser.tab.h in the build directory. Flex is invoked on lexer.l to generate lex.yy.c in the build directory and finally gcc compiles all source files together with the appropriate include paths to produce the yeast binary in the bin directory.

The makefile also provides a make clean target to remove all generated files and a make run target that accepts a file argument and runs the binary on the specified input.

4. BNF Grammar

The formal grammar of YU Assembly is defined using BNF notation. The grammar precisely specifies every valid construct in the language and the bison parser was derived from this grammar.

```
<program>          ::= <statement-list>

<statement-list>   ::= <statement>
                    | <statement-list> <statement>

<statement>        ::= <labeled-statement>
                    | <instruction>

<labeled-statement> ::= <LABEL> ":" <instruction>

<instruction>       ::= <mov-instr> | <load-instr> | <store-instr>
                    | <addi-instr> | <add-instr> | <and-instr>
                    | <or-instr>  | <xor-instr> | <blt-instr>
                    | <bgt-instr> | <beq-instr>

<mov-instr>        ::= "mov" <register> "," <immediate>
<load-instr>       ::= "load" <register> "," <mem-addr>
<store-instr>      ::= "store" <register> "," <mem-addr>
<addi-instr>       ::= "addi" <register> "," <register> "," <immediate>
<add-instr>        ::= "add" <register> "," <register> "," <register>
<and-instr>        ::= "and" <register> "," <register> "," <register>
<or-instr>         ::= "or" <register> "," <register> "," <register>
<xor-instr>        ::= "xor" <register> "," <register> "," <register>
<blt-instr>        ::= "blt" <register> "," <register> "," <LABEL>
<bgt-instr>        ::= "bgt" <register> "," <register> "," <LABEL>
<beq-instr>        ::= "beq" <register> "," <register> "," <LABEL>

<mem-addr>         ::= "mem[" <register> "]"
                    | "mem[" <register> "+" <immediate> "]"
                    | "mem[" <register> "-" <immediate> "]"

<register>          ::= "R0" | "R1" | "R2" | "R3"
                    | "R4" | "R5" | "R6" | "R7"

<immediate>        ::= <NUMBER> | "-" <NUMBER>

<LABEL>            ::= <LETTER> { <LETTER> | <DIGIT> | "_" }
<NUMBER>           ::= <DIGIT> { <DIGIT> }
```

Several design decisions were made in the grammar to enforce the language rules from the specification. The register non-terminal enumerates only R0 through R7 explicitly so R8 and above are never recognized as valid registers. The branch instructions take a LABEL as their third operand rather than a register, which enforces the syntactic distinction between jump targets and data operands. The mov instructions second alternative explicitly matches the case where a label is provided and immediately produces an error, making the rejection message informative rather than generic.

5. Lexical Analysis

5.1 Token Definitions

The lexer is implemented in `lexel.l` using Flex. It recognizes all tokens required by the YU Assembly grammar and is configured with several options to improve integration with the parser. The `noyywrap` option removes the need for a multi-file input function. The `yylineno` option instructs Flex to maintain a running line counter automatically, which is used in all error messages. The `nounput` and `noinput` options suppress warnings about unused internal Flex functions.

The complete set of tokens recognized by the lexer is described below:

Token	Pattern	Description
MOV, LOAD, STORE,	Keyword strings	One token per instruction keyword
MEM	"mem"	Memory addressing keyword
REGISTER	R[0-7]	Valid register — stores number (0-7) in <code>yylval.ival</code>
NUMBER	[0-9]+	Positive integer — stores value in <code>yylval.ival</code>
LABEL	[a-zA-Z][a-zA-	Identifier — stores string copy in <code>yylval.sval</code>
COMMA	","	Operand separator
COLON	":"	Label definition marker
LBRACKET	"["	Memory address open bracket
RBRACKET	"]"	Memory address close bracket
PLUS / MINUS	"+" / "-"	Arithmetic operators for offsets and immediates
NEWLINE	"\n"	Statement terminator

5.2 Lexer Implementation:

A critical design decision in the lexer is the ordering of rules. Flex matches rules from top to bottom on a tie, so keyword rules (`mov`, `load`, `store`, etc.) must appear before the generic LABEL rule. Without this ordering, instruction keywords would be tokenized as label identifiers and the parser would fail.

Newlines are returned as a dedicated NEWLINE token rather than being silently skipped. This allows the parser to treat newlines as statement terminators, which is natural for an assembly language where each instruction occupies its own line. Horizontal whitespace (spaces and tabs) is silently discarded.

When a Register token is recognized, the register number is extracted by subtracting the ASCII value of '0' from the second character of the matched text. This gives an integer in the range 0-7 stored in `yylval.ival`. When a LABEL token is recognized, `strip` is used to create a heap-allocated copy of the matched text stored in `yylval.sval`, ensuring the string remains valid after the next token is read.

5.3 Error Handling

The lexer implements two levels of error handling. For invalid registers, a dedicated rule matches the pattern R followed by any sequence of digits. Since Flex applies the longest match rule, this pattern matches R8, R9 R10 and any other out of range register name before they cause an error. When this happens, the lexer prints an error message. For any character that matches none of the defined patterns, the catch all rule works and prints the unknown character. This ensures that no input character is silently ignored.

6. Syntax Analysis

6.1 Grammar Rules

The parser is implemented in `parser.y` using Bison. It is driven by the token stream produced by the lexer and validates the input program against the YU assembly grammar. On success it produces a fully constructed abstract syntax tree. On failure, it produces a descriptive error messages that include the line number where the errors occurred.

The Bison grammar mirrors the BNF specification closely. The top-level program rule reduces a statement-list and calls `set-program-root` to store the root AST node in a global variable accessible from `main.c`. The statement-list rule is left-recursive, allowing programs of arbitrary length to be parsed correctly.

Individual instruction rules are each defined separately. This verbosity is intentional - it allows each instruction to have its own semantic action that calls the correct AST node constructor with the right operands.

6.2 Parser Actions

Each grammar rule in the parser has an associated semantic action enclosed in curly braces. These actions are responsible for constructing the AST nodes that represent the parsed program. The parser builds the tree bottom-up: leaf nodes are created first, then instruction nodes are created from their operands and finally statement and program nodes are created to link instructions together.

The union directive at the top of the parser file declares three value types: an integer (`ival`) for register numbers and immediate, a character pointer (`eval`) for label names and an `ASTNode` pointer for all grammar rules that produce tree nodes. Every token and non-terminal is associated with the correct union member through token type declarations.

Memory management for label strings is handled carefully within the parser. When a labeled statement is parsed, the label string returned by the lexer is passed to `make-label-def-node` which calls `strip` internally. The original string is then freed with `free($1)` to avoid a memory leak.

6.3 Error Detection

The parser enforces several error conditions beyond what is captured by the grammar alone. The most important is the handling of label-as-mov-operand. The grammar includes a second alternative for the `mov-instr` rule that explicitly matches `MOV LABEL COMMA immediate`. When this alternative fires, the parser prints a targeted error message stating that `mov` requires a register, not a label and calls `YYERROR` to abort parsing.

Missing commas are naturally detected as syntax errors by the parser because every instruction rule explicitly includes `COMMA` tokens between operands. If a comma is absent, the parser fails to match the rule and calls `terror` with the current line number.

The `terror` function at the bottom of the parser file is called by Bison whenever a syntax error is encountered. It prints a message of the form "syntax error at line ..." to standard error, providing the user with enough information to locate and correct the problem.

7. Abstract Syntax Tree

7.1 Node Types

The abstract syntax tree is the central data structure of the compiler. It is defined in `ast.h` and implemented in `ast.c`. Every construct in a YU assembly program. Instructions, labels, memory addresses and the program structure itself, is represented as a node in the tree.

The `NodeType` enumeration defines sixteen distinct node types:

Node Type	Represents	Key Fields
NODE_PROGRAM	List node linking statements	left, right (ASTNode*)
NODE_LABELED_STMT	Label definition + instruction	label, instr (ASTNode*)
NODE_LABEL_DEF	Label name	name (char*)
NODE_MOV	mov instruction	rd (int), imm (int)
NODE_LOAD	load instruction	reg (int), mem (ASTNode*)
NODE_STORE	store instruction	reg (int), mem (ASTNode*)
NODE_ADDI	addi instruction	rd, rs (int), imm (int)
NODE_ADD	add instruction	rd, rs1, rs2 (int)
NODE_AND	and instruction	rd, rs1, rs2 (int)
NODE_OR	or instruction	rd, rs1, rs2 (int)
NODE_XOR	xor instruction	rd, rs1, rs2 (int)
NODE_BLT	blt instruction	rs1, rs2 (int), label (char*)
NODE_BGT	bgt instruction	rs1, rs2 (int), label (char*)
NODE_BEQ	beq instruction	rs1, rs2 (int), label (char*)
NODE_MEM	Memory address	reg (int), offset (int), sign (int)

7.2 AST Construction

Each node type has a dedicated constructor function. All constructors follow the same pattern: they call `alloc_node` which uses `calloc` to allocate a zeroed `ASTNode`, set the type field, fill in the data union fields from the function parameters and return the pointer. Using `calloc` ensures that all pointer fields are initialized to `NULL`, preventing accidental use of uninitialized memory.

The arithmetic instructions (add, and, or, xor) all share the same data layout the union (rd,rs1,rs2) and are handled by a single internal `make-alu-node` function. The four public constructors `make-add-node`, `make-and-node` and `make-xor-node` each call this internal function with the appropriate `NODE*_` type eliminating code duplication.

The program is represented as a linked list of statement nodes using the `NODE_PROGRAM` nodes left and right pointers. The left pointer points to the first statement and the right pointer points to another `NODE_PROGRAM` node containing the remainder of the program.

8. Loop Unrolling Optimization

8.1 Algorithm Overview

Loop unrolling is a classical compiler optimization technique that reduces loop overhead by replicating the loop body. The motivation is straightforward: every loop iteration requires a conditional branch instruction and one or more counter-increment instructions by executing two iterations per branch instead of one, the number of branch and increment operations is cut in half, reducing both instruction count and branch prediction pressure.

The loop unrolling pass is implemented in unroll.c and operates in five stages:

1. Flatten — The AST is traversed recursively and all statement nodes are collected into a linear array. This converts the tree representation into an indexed sequence, making it possible to reason about relative positions of instructions.
2. Label Map — The flat array is scanned for labeled statements and their names and indices are recorded in label map. This data structure enables O(n) lookup at any labels position.
3. Backward Branch Detection — The flat array is scanned for branch instructions. For each branch, its target label is looked up in the label map. If the targets index is less than the branches index, this is a backward branch and therefore indicates a loop.
4. Body Duplication — The loop body is reconstructed in the result array with body appearing twice, memory offsets adjusted in the duplicate, addi immediate doubled and the branch instruction appearing once at the end.
5. AST Reconstruction — The modified flat array is passed to build_ast which reassembles it into a new AST using make-program_node calls.

8.2 Backward Branch Detection

A loop is defined in the project specification as a sequence of instructions containing exactly one backward branch instruction whose target label was defined earlier in the program. The detection algorithm is therefore simple: for each branch instruction at index I, if its target label has index loop_start where loop_start < I, a loop has been found with the body spanning indices loop_start inclusive.

This definition correctly handles programs with multiple independent loops by running the detection scan for every branch in the program. It also correctly handles programs with no loops by simply not modifying the flat array.

8.3 Example

The following example illustrates the transformation applied to the sample program from the project handout:

mov R2, 0	mov R2, 0	
mov R3, 100	mov R3, 100	
mov R4, 50	mov R4, 50	
L: load R1, mem[R2]	L: load R1, mem[R2]	<- copy 1
store R1, mem[R3]	store R1, mem[R3]	<- copy 1
	load R1, mem[R2+1]	<- copy 2 (offset +1)
	store R1, mem[R3+1]	<- copy 2 (offset +1)
addi R2, R2, 1	addi R2, R2, 2	<- doubled (1 -> 2)
addi R3, R3, 1	addi R3, R3, 2	<- doubled (1 -> 2)
blt R2, R4, L	blt R2, R4, L	<- unchanged

Input Program

Output Program after Unrolling by Factor of 2

9. Symbol Table — Bonus

9.1 Purpose

The symbol table is implemented as a bonus component providing semantic analysis beyond what the parser guarantees. While the parser validates that branch instructions are syntactically correct, it cannot verify that the label referenced by a branch instruction was actually declared somewhere in the program. The symbol table fills the gap.

Consider the following program:

```
mov R0, 0
mov R1, 10
blt R0, R1, UNDEFINED_LABEL    # This label was never declared
```

This program passes lexical and syntax analysis without error. The parser sees a syntactically valid bit instruction with two registers and a label token. But at runtime the branch would have no valid target. The symbol table detects this condition and report it as a semantic error before the optimization pass run.

9.2 Two-Pass Algorithm

The symbol table check is implemented as a two-pass algorithm over the AST:

Pass 1 - Label Collection:

The AST is traversed recursively. Every node-labeled-stmt node encountered has its label name extracted and added to the symbol table along with its index in the program. After this pass, the table contains an entry for every declared label in the program.

Pass 2 - Branch Validation:

The AST is traversed again. Every branch instruction node has its target label looked up in the symbol table. If the lookup returns 0, a semantic error is printed. The function accumulates error counts across the entire program, so all undefined labels are reported in a single pass rather than stopping at the first error.

The check-labels function returns 0 if no errors were found or -1 if any errors were detected. In main.c, a non zero return value causes the compiler to exit.

9.3 Implementation Details

The symbol table is implemented as a array of symbol structs, each holding a label name and an index. The table starts with an initial capacity of 16 entries and doubles in capacity when full using realloc.

The symtable-lookup function performs a scan through entries array comparing label names with strcmp. A hash table could be used if performance were a concern but the simplicity of the scan was preferred for this implementation.

The symtable_print function is called at the start of the symbol table check and prints a summary of all declared labels and their positions. This provides useful debugging output and also demonstrates to the marker that the symbol table was built correctly before validation begins.

10. Test Cases

Six test programs were prepared to demonstrate the compilers behavior across a range of scenarios. Three are valid programs that should be accepted and optimized, three are valid and three are unvalid.

10.1 Valid Programs

valid1.asm — Basic Copy Loop (handout example)

This program copies 50 values from memory locations [0..49] to memory locations [100..149] using a counted loop.

```
mov R2, 0
mov R3, 100
mov R4, 50
L: load R1, mem[R2]
store R1, mem[R3]
addi R2, R2, 1
addi R3, R3, 1
blt R2, R4, L
```

Expected Output:

The symbol table check finds one entry ([3] L) and confirms all branches are resolved. The unrolled program duplicates the load/store body with offset +1 and doubles the addi immediate from 1 to 2, exactly matching the handouts expected output.

Actual Output:

```
legehuriel@192 Yu-Assembly % make run FILE=tests/valid1.asm

-----Running-----

./bin/yuasm tests/valid1.asm
PARSING -- tests/valid1.asm ...
parsing successfull

-----Symbol Table Check-----
symbol table (1 entries):
[3] L
```

```
Applying loop unrolling...

=== Unrolled Program ===
mov R2, 0
mov R3, 100
mov R4, 50
L:
load R1, mem[R2]
store R1, mem[R3]
load R1, mem[R2+1]
store R1, mem[R3+1]
addi R2, R2, 2
addi R3, R3, 2
blt R2, R4, L
```

valid2.asm — Arithmetic Instructions, no loop

This program exercises the arithmetic and bitwise instructions without any branching or loops. It verifies that programs without loops are accepted and pass through the optimization pass unchanged.

```
mov R0, 5
mov R1, 3
add R2, R0, R1
and R3, R0, R1
or R4, R0, R1
xor R5, R0, R1
```

Expected Output:

Symbol table is empty. No branch target to validate. Unrolled program is identical to the input.

Actual Output:

```
legehuriel@192 Yu-Assembly % make run FILE=tests/valid2.asm

-----Running-----

./bin/yuasm tests/valid2.asm
PARSING -- tests/valid2.asm ...
parsing successfull

-----Symbol Table Check-----
symbol table (0 entries):
```

```
Applying loop unrolling...

=== Unrolled Program ===
mov R0, 5
mov R1, 3
add R2, R0, R1
and R3, R0, R1
or R4, R0, R1
xor R5, R0, R1
```

valid3.asm — Multiple Labels and nested loops

This program contains two independent counted loops with different labels, testing that the compiler correctly handles multiple backward branches.

```
mov R0, 0
mov R1, 10
LOOP1: addi R0, R0, 1
blt R0, R1, LOOP1
mov R2, 0
mov R3, 5
LOOP2: addi R2, R2, 1
blt R2, R3, LOOP2
```

Expected Output:

Symbol table contains two entries: LOOP1 and LOOP2. Both branch targets are validated successfully. The first loop is unrolled with addi doubled from 1 to 2 and the second loop is similarly unrolled.

Actual Output:

```
legehurriel@192 Yu-Assembly % make run FILE=tests/valid3.asm
-----Running-----

./bin/yuasm tests/valid3.asm
PARSING -- tests/valid3.asm ...
parsing successfull

-----Symbol Table Check-----
symbol table (2 entries):
[2] LOOP1
[6] LOOP2
```

```
=== Unrolled Program ===
mov R0, 0
mov R1, 10
LOOP1:
addi R0, R0, 1
addi R0, R0, 1
blt R0, R1, LOOP1
mov R2, 0
mov R3, 5
LOOP2:
addi R2, R2, 1
blt R2, R3, LOOP2
```

10.2 Invalid Programs

invalid1.asm — Out of range register

```
mov R8, 5
```

Expected Output → Lexer error at line 1

invalid2.asm — Missing Comma

```
mov R0 5
```

Expected Output → Syntax error at line 1

invalid3.asm — Label in mov operand

```
mov L, 5
```

Expected Output → Syntax error at line 1

11. Compilation and Execution

The project can be compiled and executed using standard GCC, Flex and Bison tools available on any Linux, Windows or macOS system. The makefile automates the entire build process.

Prerequisites:

- GCC
- Flex
- Bison

Build:

- *make all*

This runs Bison on `src/parser.y` outputting to `build/`, then Flex on `src/lexer.l` outputting to `build/`, then GCC on all source files producing `bin/yuasm`.

Run on a test file:

- *make run FILE=test/valid1.asm*
- *./bin/yuasm tests/valid1.asm*

Clean build artifacts:

- *make clean*

The program reads the specified YU assembly source files, runs the full pipeline (parse, symbol table check, loop unrolling) and prints the optimized program to standard output. Any errors detected during flexing, parsing or semantic analysis are printed to standard error and cause the program to exit

12. Conclusion

This project resulted in a complete and functional compiler front-end for the YU Assembly language. All three main tasks defined in the project handout were successfully implemented: the BNF grammar formally specifies the language syntax, the Flex lexer and Bison parser correctly accept valid programs and rejects invalid ones with informative messages and the loop unrolling optimization produces output that exactly matches the expected results from the handout examples.

The bonus symbol table implementation adds a meaningful layer of semantic correctness checking that goes beyond what the parser alone can verify. By performing two passes over the AST, one to collect all declared labels and one to validate all branch targets, the compiler catches undefined label references that would otherwise only be discovered at runtime.

The project also provided practical experience with several important concepts for programming languages: the role of lexical and syntax analysis in a compiler pipeline, the design and use of an abstract syntax tree as a central intermediate representation, the integration of multiple tools in a single build system and the application of a classical optimization technique to a structured program representation.

Working through the debugging process, particularly the Makefile tab issues, the Flex indentation rules and the loop unrolling ordering, reinforced the importance of understanding not just the high level concepts but also the low-level mechanics of each tool.

13. References

- CSE 351 Programming Languages Lecture Slides and Notes — Yeditepe University, 2026
- Stack Overflow, Question on Flex/Bison integration, C memory management <https://stackoverflow.com>
- Loop Unrolling Wikipedia https://en.wikipedia.org/wiki/Loop_unrolling
- Loop Unrolling GeeksforGeeks <https://www.geeksforgeeks.org/c/loop-unrolling/>
- Symbol Table Implementation <https://home.adelphi.edu/~siegfried/cs372/372l3.pdf>
- Debugging & Error Handling — ChatGPT & StackOverflow
- CSE 351 Programming Languages - Term Project Instruction PDF - Yeditepe University, 2026